Introdução à Lógica e Programação

Sequenciação, variáveis e operações aritméticas

Olá Mundo!

O comando **`print()`** gera uma impressão no terminal. O texto a ser impresso deve ser informado dentro do parênteses e entre aspas simples ou aspas duplas.

```
print('Olá mundo!')
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

/bin/python3 /home/alexandre/Documentos/temp/python/main.py
alexandre@alexandre-Inspiron-15-5510:~/Documentos/temp/python$ /bin/python3 /home/
alexandre/Documentos/temp/python/main.py
Olá Mundo!
alexandre@alexandre-Inspiron-15-5510:~/Documentos/temp/python$
```

Imprimindo números

O comando **`print()`** também aceita valores numéricos inteiros ou reais. Neste caso não é necessário usar aspas no conteúdo a ser impresso.

Repare que o símbolo de ponto é usado como separador decimal ao invés da vírgula.

```
print(12)
print(6.5)
print(4.2)
```

Sequenciação

Um programa de computador é composto por um conjunto de **instruções**, as quais são executadas uma por vez e em sequência.

A ordem de execução das instruções é determinada pela sequência de ocorrência das instruções no código-fonte. Para simplificar: a primeira linha contém a primeira instrução, a segunda linha contém a segunda instrução e assim por diante.

```
print("Olá!")
print("Seja bem-vindo")
print("Esta é uma mensagem")
print("Esta é outra mensagem")
```

Exercício 1

Escreva um programa que gere a impressão indicada abaixo.

```
Estudar no IFRN é demais.
Cursar InfoWeb é melhor ainda.
Programar em Python é incrível!
```

Exercício 2

Escreva um programa que imprime a seguinte sequência de informações, cada uma em um linha do terminal:

- Seu nome
- Sua matrícula no IFRN
- Em que cidade e bairro você mora
- Sua(s) matéria(s) favorita(s)

Variável

Um programa de computador trabalha com dados que precisam estar facilmente disponíveis. Uma **variável** é uma área de memória utilizada pelo programa para guardar e acessar um dado de forma simples.

Uma variável possui um nome, também chamado de **identificador**, e um **tipo de dado**. Além disso, o valor (dado) da variável pode ser modificado ao longo da execução do programa. No entanto, uma variável só pode armazenar um valor por vez.

Variável

No código abaixo, **x** e **y** são variáveis.

Repare que a variável **x** tem seu valor alterado na terceira linha do código.

```
x = 10
print(x)
x = 20
print(x)
y = 10.8
print(y)
```

Variável

Os comandos **`print()`** do programa abaixo geram a mesma impressão?

```
x = 10
print(x)
print('x')
```

Variável

Existem regras para o identificador de uma variável:

- São permitidas letras maiúsculas e minúsculas e o caractere de underscore **_**
- São permitidos dígitos, exceto para o primeiro caractere

Exemplos de identificadores válidos: **`x`**, **`media`**, **`ano`**, **`Idade`**, **`v1`**, **`v_2`**, **`_v`**, **`MAX`**

Exemplos de identificadores inválidos: **`2x`**, **`medi@`**, **`v-1`**

Python é *case sensitive*, ou seja, há distinção entre caracteres maiúsculos e minúsculos. Na prática, os identificadores **`valor`** e **`Valor`** são considerados diferentes.

Palavras reservadas

As seguintes palavras representam instruções da linguagem Python e portanto não podem ser utilizadas como identificadores de variáveis.

`False`	`await`	`else`	`import`	`pass`
`None`	`break`	`except`	`in`	`raise`
`True`	`class`	`finally`	`is`	`return`
`and`	`continue`	`for`	`lambda`	`try`
`as`	`def`	`from`	`nonlocal`	`while`
`assert`	`del`	`global`	`not`	`with`
`async`	`elif`	`if`	`or`	`yield`

Comentários

Comentários não são interpretados pelo interpretador. Eles são utilizados para descrever ou documentar o código-fonte. Comentários de uma linha são definidos com o caractere **#** (cerquilha). Tudo o que estiver à direita da cerquilha é ignorado pelo interpretador Python.

```
x = 10
print(x) # imprime 10
x = 20   # altera o valor de x
print(x)
y = 10.8
# o próximo comando imprime o valor de y
print(y)
```

Tipos de dados numéricos

Qualquer dado (valor) armazenado em uma variável possui um tipo. Neste momento, vamos trabalhar com os seguintes tipos numéricos:

- `int`: suporta valores inteiros.
- `float`: suporta valores reais.

```
x = 10     # int
y = 4.5    # float
z = 5.0    # float
```

Operadores aritméticos

São utilizados para a realização de cálculos com dados numéricos.

Operação	**Operador**	**Exemplo**	**Resultado**
Adição	+	5 + 3	8
Subtração	-	3 - 8	-5
Multiplicação	*	1.2 * 10	12.0
Divisão real	/	10 / 4	2.5
Divisão inteira	//	10 // 4	2
Módulo (resto da divisão)	%	7 % 2	1
Potenciação	**	3 ** 2	9

Operadores aritméticos

Potência

3 ** 2 = 3^2 = 9

Divisão real

10 / 4 = 2.5

```
 10 | 4
-  8   2.5
   20
 - 20
      0
```

Divisão inteira

10 // 4 = 2

```
 10 | 4
-  8   2
   2
```

Módulo

(resto da divisão inteira)

7 % 2 = 1

```
  7 | 2
- 6   3
  1
```

Operadores aritméticos

```
x = 5 + 3
print(x)   # imprime 8
x = 3 - 8
print(x)   # imprime -5
x = 1.2 * 10
print(x)   # imprime 12
x = 10 / 4
print(x)   # imprime 2.5
x = 10 // 4
print(x)   # imprime 2 (considera apenas a parte inteira do resultado)
x = 7 % 2
print(x)   # imprime 1 (resto da divisão de 7 por 2)
x = 3 ** 2
print(x)   # imprime 9
```

Operadores aritméticos

Podemos usar variáveis e literais como operandos dos operadores.

```
x = 5
y = 3
z = x + y
print(z)   # imprime 8
z = 3 - x
print(z)   # imprime -2
z = 1.5 * y
print(z)   # imprime 4.5
z = x + y + 4.0
print(z)   # imprime 12.0 (repare que o resultado é do tipo float)
```

Precedência de operadores

Define a ordem das operações em uma expressão com mais de um operador aritmético. Parênteses podem ser utilizados para alterar a ordem das operações. Os parênteses mais internos são avaliados primeiro. Em caso de empate, a avaliação é realizada da esquerda para a direita.

Precedência	Operadores
1º	( )
2º	**
3º	* / // %
4º	+ -

Precedência de operadores

- 5 + 3 * 2 ⇒ 11
- 10 / 2 * 3 ⇒ 15
- 16 / 2 ** 3 * 5 ⇒ 10
- (5 + 3) * 2 ⇒ 16
- (16 / (2 + 6)) * 3 ⇒ 6

Exercícios

3) Escreva um programa que imprima seu nome dentro de uma caixa da seguinte maneira:

```
+------+
|Paulo|
+------+
```

4) Escreva um programa que calcula e imprime a soma dos inteiros de 1 a 10.
5) Escreva um programa que imprime a soma das áreas de um quadrado cujo lado mede 2 e de outro quadrado cujo lado mede 3. Use uma única expressão aritmética para calcular o valor da soma.
6) Escreva um programa que calcula e imprime a média aritmética de três números reais de sua escolha.
7) Escreva um programa que imprime o resultado da multiplicação de 3 por 1.2. O resultado impresso está matematicamente correto? Qual é a explicação para o valor obtido?

Exercícios

8) Qual é o resultado de cada uma das expressões abaixo?

a) 3 + 6 - 10
b) 2 + 2 * 4
c) 5 * 3 - 2 * -5
d) 30 - 20 + 3 ** 3
e) 15 // 6 * 5 % 3
f) 2 ** 5 + 5
g) 6 % 10
h) 2 ** (1 ** 10 + 3) // 5
i) - 5 * (3 + (- 3 - 8) + 1) * -1

Strings

String é o tipo de dado que permite trabalharmos com texto (a palavra string pode ser traduzida como sequência de caracteres).

Em Python, valores do tipo string são criadas através de aspas simples ou aspas duplas.

Strings podem ser concatenadas (unidas) através do operador +. O resultado é sempre uma nova string.

```
print('Olá mundo!')
a = "IFRN"
print(a)
b = 'Eu estudo no ' + a
print(b)
```

Repetição de strings

Podemos usar o operador * para gerar uma repetição de string.

```
s = "A" * 5
print(s)
s = 'ba' + 'na' * 2
print(s)
print('-' * 10)
```

Caracteres de escape

Os seguintes caracteres não podem ser digitados em uma string pois possuem significado especial: aspas simples, aspas duplas e contrabarra. Caracteres de escape devem ser utilizados quando precisamos inserir tais caracteres em uma string.

Também utilizamos caracteres de escape para inserir caracteres tais como tabulação e quebra de linha.

```
print('Olá \n mundo!')  # \n é o caractere de escape de quebra de linha
print('Olá \t mundo!')  # \t é o caractere de escape de tabulação
print('Olá \'mundo\'!') # \' é o caractere de escape de aspas simples
print('Olá \"mundo\"!') # \" é o caractere de escape de aspas simples
print('Olá \\ mundo!')  # \\ é o caractere de escape de contrabarra
```

F-strings

É comum que queiramos imprimir texto em conjunto com valores presentes em variáveis. Podemos realizar isso através de F-strings (strings precedidas do caractere f).

```
a = 3
print(f'O bebê tem {a} anos')
b = 2.5
print(f"Ele comprou {b} kg de farinha")
c = 'dia'
print(f'Bom {c}!')
print(f"TOTAL = {a + b}")
d = 3.666666
print(f'{d:.2f}')
```

Quantidade de caracteres de uma string

Podemos usar o comando `len()` para obter a quantidade de caracteres de uma string.

```
a = "IFRN"
b = len(a)
print(b)
b = len("Olá Mundo!")
print(b)
```

Comando `input`

Usamos o comando `input` para capturar entradas realizadas pelo teclado (ou seja, para capturar a digitação do usuário).

O comando `input` sempre entrega a entrada como uma string. Usamos os comandos `int()` e `float()` para converter strings em valores numéricos.

```
a = input()
b = a * 3    # b armazena uma repetição de string
print(b)
a = int(a)
b = a * 3    # b armazena um valor inteiro
print(b)
# Lendo um valor real:
c = input()
c = float(c)
# Em um único passo:
c = float(input())
```

Exercícios

9) Escreva um programa que tem como entrada um nome e que o imprime junto com 10 caracteres de asterisco à sua esquerda e à sua direita como no exemplo a seguir:

```
********** Paulo **********
```

10) Escreva um programa que tem como entrada um nome e o imprime dentro de uma caixa da seguinte maneira:

```
+-----+
|Paulo|
+-----+
```

11) Escreva um programa que tem como entrada um número inteiro e imprime o seu dobro.
12) Escreva um programa que calcula a multiplicação de 3 por 1.2 e imprime o resultado usando apenas uma casa decimal à direita do ponto.
13) Escreva um programa que tem como entrada 3 números inteiros e imprime a soma desses números.
14) Escreva um programa que tem como entrada 4 números reais e imprime a média aritmética desses números. Na impressão, use apenas uma casa decimal à direita do ponto.

Referências

ASCENCIO, Ana F. G.; DE CAMPOS, Edilene A. V.; **Fundamentos da programação de computadores**: Algoritmos, Pascal, C/C++ e Java. 2ª ed. São Paulo: Pearson Prentice Hall, 2007.

Python Software Foundation. **The Python Language Reference**. Disponível em <https://docs.python.org/3/reference/index.html>. Acesso em 22/02/2024.

HORSTMAN, Cay. **Conceitos de computação com o essencial de Java**. 3ª ed. Porto Alegre: Bookman, 2005.

Informações bibliográficas

Autor: Alexandre Gomes de Lima

Data: abril de 2025.

Direitos autorais: CC BY 4.0
(https://creativecommons.org/licenses/by/4.0/deed.pt-br)